

МЕТОДЫ И СТАНДАРТЫ ПРОГРАММИРОВАНИЯ

СЕМИНАРЫ

II семестр

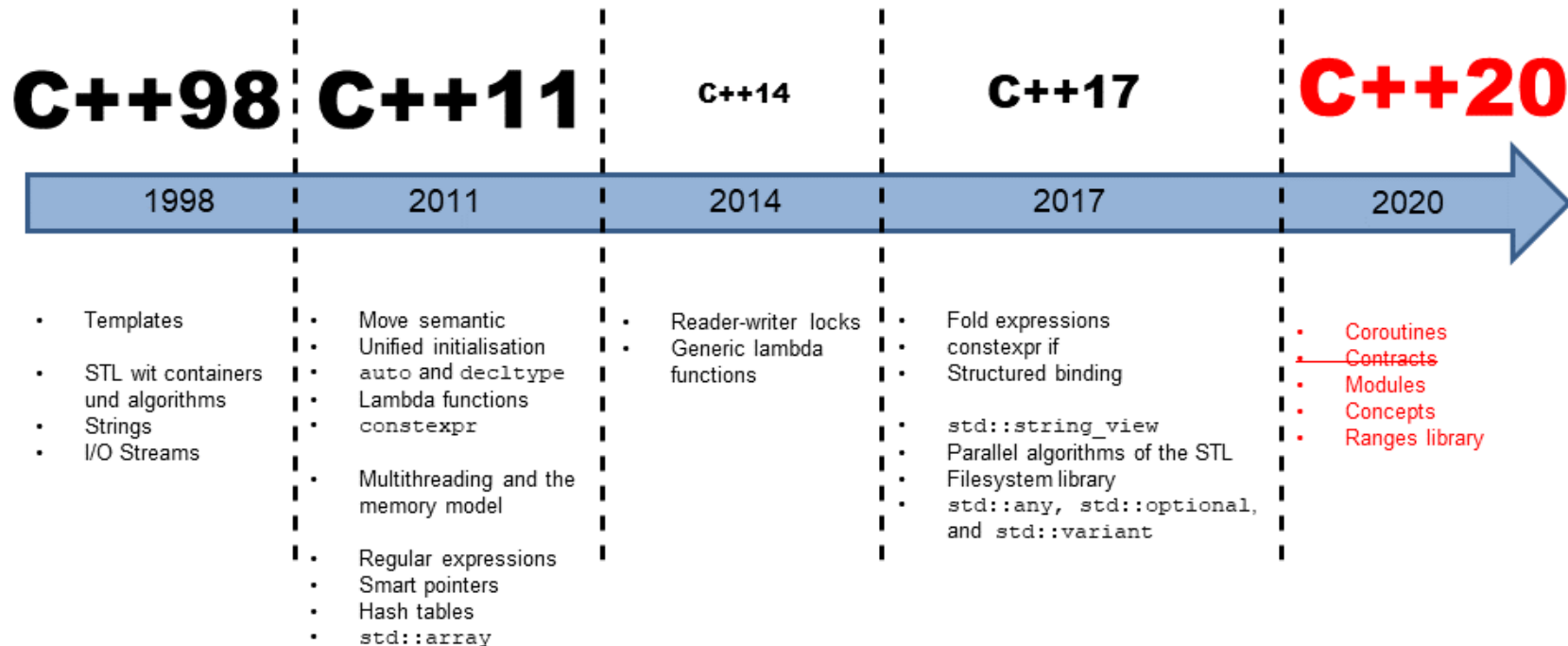
С И С++

Семинар 1

Совместимость Си и С++

- С++ создавался как потомок до стандартизированного С, по большей части совместимый с ним на тот момент на уровне исходного кода и компоновки.
- Средства разработки для обоих языков (такие, как среды разработки и компиляторы) часто интегрируются в один продукт.
- С не является подмножеством С++, поэтому нетривиальные программы на С не будут компилироваться на С++ без изменений.
- С++ вводит множество возможностей, недоступных в С, и на практике почти весь код, написанный на С++, не соответствует коду на С.

Стандарты C++



Общие различия

С	С++
Код читается проще	Код выглядит сложнее
Проще портировать код на другие платформы	Из-за особенностей языка код портировать сложнее
Есть структуры и переменные типа «структура»	Есть классы и объекты
Наследование — это структура внутри структуры	Классическое наследование в стиле ООП
void — обязательное слово	Использовать слово void не обязательно
Можно объявлять глобальную переменную несколько раз	Глобальная переменная объявляется только один раз
Исключения нужно конструировать самому	Есть обработка исключений
Нет перегрузки операторов	Операторы можно перегружать
Почти всё стандартно и предсказуемо	Легко написать код, в котором непонятно, где ошибка

Типы данных

Запрещается делать массивы переменной длины (Variable Length Arrays, VLA).

Для перечислений в C это — тип `int`, в C++ — это тип самого перечисления.

Приведение типов

Язык C++ сильнее типизирован, нежели C, т.е. данные конструкции скомпилируются на C, а вот на C++ нет.

```
void* ptr;  
int* i_ptr = ptr;  
  
int* imass = malloc(128 * sizeof(int));
```

В C++ необходимо использовать явное приведение типа.

```
int* imass = (int*)malloc(128 * sizeof(int));
```

но в таком варианте приведения компилятор выдаст предупреждение $\Rightarrow$

Warning: use of old-style cast

Данное предупреждение выдает компилятор на приведение вида:

```
int* imass = (int*)malloc(128 * sizeof(int));
```



```
int* imass = static_cast<int*>(malloc(128 * sizeof(int)));
```

Для корректного приведения типов есть следующие операторы приведения:

- `static_cast<T>(e)` – преобразование выражения одного статического типа в значение другого статического типа;
- `const_cast<T>(e)` – отбросить квалификатор `const` у изначально не константных данных или добавить квалификатор `const`;
- `reinterpret_cast<T>(e)` – приведение типов без проверки, т.е. непосредственное указание компилятору, используется только в случае полной уверенности программиста в собственных действиях;
- `dynamic_cast<T>(e)` – преобразует тип данных (указатель или ссылку) вниз по иерархии наследования (из типа-предка в тип-потомок), с проверкой с помощью динамической идентификации типа данных.

Функции стандартной библиотеки C

C++ изменяет некоторые функции стандартной библиотеки языка Си, добавляя дополнительные перегруженные функции с квалификатором типа `const`, например, `strchr` возвращает `char*` в C, в то время как C++ поступает так, как если бы существовали две перегруженные функции `const char *strchr(const char *)` и `char *strchr(char *)`.

Объекты с квалификатором const

```
typedef struct  
{  
    const int i = 65;  
} Dot;
```

C – ✗

Инициализировать
можно только при
создании объекта!

C++ – ✓

Символьные литералы

Символьные литералы, такие как 'a', имеют тип `int` в C и тип `char` в C++, это означает, что `sizeof 'a'` обычно даёт разные результаты на двух языках: в C++ это будет 1, в то время как в C это будет `sizeof(int)`. Как ещё одно следствие этого различия в типах, в C 'a' всегда будет выражением со знаком, независимо от того, является `char` знаковым или беззнаковым, тогда как для C++ это зависит от реализации компилятора.

Логический тип `bool`

И C99, и C++ имеют логический тип `bool` с константами `true` и `false`, но они определены по-разному. В C++ `bool` — это встроенный тип и зарезервированное ключевое слово.

В C99 новое ключевое слово `_Bool` вводится как новый логический тип. Заголовок `stdbool.h` содержит макросы `bool`, `true` и `false`, которые определены как `_Bool`, 1 и 0, соответственно. Следовательно, `true` и `false` имеют тип `int` в C.

Программа на языке C++

```
// подключаем заголовочный файл iostream
#include <iostream>
int main() // определяем функцию main
{
    // выводим строку на консоль
    std::cout << "Hello World!" << std::endl;
    return 0;
}
```

Изменения по сравнению с Си и работой в прошлом семестре

- Компилятор: `gcc` $\rightarrow$ `g++`
- Расширения файлов: `c` $\rightarrow$ `crr`, `h` $\rightarrow$ `hrr/h` (при этом расширение ***hrr*** используется довольно редко)

Ввод/вывод C++

Функции ввода/вывода языка C++ находятся в заголовочном файле `iostream`.

- `std::cout` – поток стандартного вывода;
- `std::cerr` – поток стандартного вывода ошибок;
- `std::cin` – поток стандартного ввода.

Для работы с потоком вывода используется оператор `<<`, с потоком ввода `>>`.

Потоки ввода/вывода работают с любыми фундаментальными типами

ВВОД/ВЫВОД C++

printf(...) → stdout → std::cout <<

scanf(...) → stdin → std::cin >>

stderr → std::cerr

```
1  #include <iostream>
2
3  int main(int argc, char** argv)
4  {
5      int i = 0;
6      double d = 0.;
7      char ch = 0;
8
9      std::cin >> i >> d >> ch;
10     std::cout << "i = " << i << " d = " << d << " ch = " << ch << std::endl;
11     std::cerr << "Error: " << i << std::endl;
12
13     return 0;
14 }
```

```
shulginaali@075-AShulgina:~/cpp$ ./a.out
19999 0.127823 6
i = 19999 d = 0.127823 ch = 6
Error: 19999
```

```
shulginaali@075-AShulgina:~/cpp$ ./a.out
f 4.3 r
i = 0 d = 0 ch =
Error: 0
```

Манипуляторы ввода/вывода

- `std::endl` – перевод строки $\sim \backslash n$;
- `std::setprecision(int)` – установка точности вывода вещественных типов (количество знаков);
- `std::scientific` – “научный” формат вывода вещественных чисел (`1.000000e-02`);
- `std::fixed` – вывод вещественного числа с фиксированной точкой;
- `std::hex` – ввод/вывод данных в шестнадцатеричной системе;
- `std::oct` – ввод/вывод данных в восьмеричной форме;
- `std::setw(int w)` – установка ширины поля равной `w`;
- Остальные можно посмотреть в документации (`<iomanip>`).

Пример использования манипуляторов IO

```
1  #include <iostream>
2  #include <iomanip>
3
4  int main(int argc, char** argv)
5  {
6      int code = 120;
7      std::cout << code << " " << std::hex << code << " " << std::oct << code << std::endl;
8      double _pi = 3.14159265359;
9      std::cout << std::setprecision(4) << _pi << " " << std::scientific << _pi << std::endl;
10     std::cout << std::fixed << std::setw(6) << std::setprecision(2) << _pi << std::endl;
11     return 0;
12 }
```

```
shulginaali@075-AShulgina:~/cpp$ ./a.out
120 78 170
3.142 3.1416e+00
  3.14
shulginaali@075-AShulgina:~/cpp$
```

Динамическое выделение памяти

- Выделение памяти:
new, new[]
- Очистка памяти:
delete, delete[]

```
1  #include <iostream>
2
3  int main(int argc, char** argv)
4  {
5      int* i = new int;
6      *i = 3;
7      std::cout << *i << std::endl;
8
9      const size_t arr_size = 10;
10     int* imass = new int[arr_size];
11     for (size_t i = 0; i < arr_size; ++i)
12     {
13         imass[i] = i * i;
14         std::cout << "[" << i << "] = " << imass[i] << std::endl;
15     }
16     delete i;
17     delete[] imass;
18     return 0;
19 }
```

Пространства имен

– это область определения переменных, типов, констант и функций, объединенная в один или несколько блоков.

Позволяет избежать конфликта имен, а также повторно использовать одни и те же именования переменных, функций и т.д.

```
namespace Name
{
    // Объявление переменных, констант, типов, функций
}
```

Пространства имен

Пространство имен может быть безымянным, что обычно используется для сокрытия каких-либо определений внутри одного модуля.

Пространства имен могут быть вложенными и повторяться в разных модулях (файлах).

```
// доступ к элементу ПИ
namespace MATH {
    double PI = 3.14;
}
std::cout << MATH::PI << std::endl;
```

using

Доступ к элементам может быть упрощен с помощью использования ключевого слова `using`, примененного либо к пространству имен целиком, либо же к отдельному его элементу:

```
// Позволит не указывать везде в коде std::  
using namespace std;  
// Позволит не указывать std:: для функции cin  
using std::cin;
```

Доступ к элементам

Элементы, объявленные вне пространства имен, принадлежат к глобальному пространству имен, и при совпадении имен доступ к ним осуществляется следующим образом:

```
void sqrt() {}           // Функция, объявленная в глобальном пространстве
имен
namespace MATH {
    void sqrt() {}      // Функция, объявленная в пространстве имен MATH
}

::sqrt();               // Вызов функции из глобального пространства имен
MATH::sqrt();           // Вызов функции из пространства имен MATH
sqrt();                 // Ошибка компиляции - неоднозначный вызов функции
```

Ссылки

В C роль ссылок играют указатели, но работать с ними не очень удобно и в C++ появилась отдельная сущность — ссылка (reference).

Ссылка – если T некоторый тип и есть переменная типа T , то переменная типа T со спецификатором $\&$ будет ссылкой на эту переменную, если она инициализирована этой переменной.

$T \ x;$

$T \ \&x = x;$ // $\&x$ это ссылка на x

Ссылки

- не могут быть NULL;
- не требуют разыменования;
- не может быть ссылки на ссылку;
- нельзя итерироваться (смещаться), так как это не область памяти.

Пример

```
1  #include <iostream>
2  // C variant with pointers
3  void swap(int* a, int* b)
4  {
5      int tmp = *a;
6      *a = *b;
7      *b = tmp;
8  }
9  // C++ variant with references
10 void swap(int& a, int& b)
11 {
12     int t = a;
13     a = b;
14     b = t;
15 }
```

```
16
17 int main(int argc, char** argv)
18 {
19     int a = 10, b = -10;
20     std::cout << "Before: " << a << " " << b << std::endl;
21     swap(&a, &b);
22     std::cout << "After: " << a << " " << b << std::endl;
23
24     std::cout << "Before: " << a << " " << b << std::endl;
25     swap(a, b);
26     std::cout << "After: " << a << " " << b << std::endl;
27     return 0;
28 }
```

```
shulginaali@075-AShulgina:~/cpp$ ./a.out
Before: 10 -10
After: -10 10
Before: -10 10
After: 10 -10
shulginaali@075-AShulgina:~/cpp$
```

Задачи

1. Написать функцию, меняющую значения двух переданных переменных местами без использования указателей
2. Написать функцию, возвращающую указатель на динамический массив значений факториала N . Значение N запрашивается у пользователя и вводится с клавиатуры. Вывести полученный массив на экран. Очистить память.

ВВЕДЕНИЕ В ООП

Семинар 2

Принципы ООП

- наследование – это процесс, посредством которого один объект может приобретать свойства другого,
- инкапсуляция – это механизм, который объединяет данные и код, манипулирующий этими данными, а также защищает и то, и другое от внешнего вмешательства или неправильного использования,
- полиморфизм – один интерфейс, много методов. Как пример: перегрузка функций.

Классы и объекты

Класс — это логическая абстракция, которая реально не существует до тех пор, пока не будет создан объект этого класса, т.е. то, что станет физическим представлением этого класса в памяти компьютера.

```
Cat cat; // создание объекта cat класса Cat
```

Конструкторы и деструкторы

Конструктор вызывается при создании объекта класса. Как правило, конструктор используется для:

- выделения памяти под объект класса;
- начальной инициализации внутренних данных класса.

Деструктор – специальный метод, который вызывается при удалении объекта.

Деструктор – это обратная по отношению к конструктору функция.

Виды конструкторов

- по умолчанию
- параметризованный
- копирования

Конструктор копирования

В C++ полная копия объекта нужна в следующих случаях:

1. В момент объявления нового объекта и его инициализации данными другого объекта с помощью оператора `=`.
2. Когда нужно передать объект в функцию как параметр-значение.
3. Когда нужно вернуть объект из функции по значению. ! Если в классе не объявлен конструктор копирования, то используется конструктор копирования, который автоматически генерируется компилятором. Этот конструктор копирования реализует побитовое копирование для получения копии объекта.

Деструктор

Имя деструктора совпадает с именем класса, перед которым следует символ '~'.

Деструктор не может иметь параметров.

При объявлении класса, можно объявить только один деструктор.

Rule of tree / rule of five

Один из основополагающих принципов C++ – RAII
(Resource Acquisition Is Initialization — “получение ресурса
есть инициализация”).

Rule of tree / rule of five

Правило трёх (rule of three, закон «Большой тройки»). Если классу необходим один из следующих методов:

- Деструктор
- Конструктор копирования
- Оператор копирующего присваивания,

то класс должен определять все три.

С выходом C++11 правило трёх превратилось в правило пяти (rule of five): если классу необходим один из следующих методов:

- Деструктор
- Конструктор копирования
- Оператор копирующего присваивания
- Конструктор перемещения
- Оператор перемещающего присваивания,

то класс должен определять все пять.

Инкапсуляция

Для типа `struct` все данные и методы по умолчанию `public`.

Для типа `class` все данные и методы по умолчанию `private`.

В C++ доступно несколько спецификаторов, и они изменяют доступ к данным следующим образом:

- публичные (`public`) – данные доступны всем;
- защищенные (`protected`) – доступны только классу и дочерним классам;
- приватные (`private`) – доступны только классу которому они принадлежат.

Инкапсуляция

```

1  #include <iostream>
2
3  class Contact
4  {
5  private:
6      int mobile_number;           // private variable
7      int home_number;            // private variable
8
9  public:
10     Contact()                    // constructor
11     {
12         mobile_number = 12345678;
13         home_number = 87654321;
14     }
15     void print_numbers()
16     {
17         std::cout << "Mobile number: " << mobile_number;
18         std::cout << ", home number: " << home_number << std::endl;
19     }
20 };

```

```

21
22  int main()
23  {
24      Contact Tony;
25      Tony.print_numbers();
26      std::cout << Tony.mobile_number << std::endl;
27      // will cause compile time error
28      return 0;
29  }

```

```

shulginaali@075-AShulgina:~/cpp$ g++ inc.cpp
inc.cpp: In function 'int main()':
inc.cpp:26:23: error: 'int Contact::mobile_number' is private within this context
   26 |         std::cout << Tony.mobile_number << std::endl;
      |                        ^~~~~~
inc.cpp:6:13: note: declared private here
    6 |         int mobile_number;           // private variable
      |         ^~~~~~
shulginaali@075-AShulgina:~/cpp$

```

Полиморфизм

В языке C++ есть возможность реализовывать два вида полиморфизма:

1. Статический. Этот вид полиморфизма достигается путём использования перегруженных функций (раннее связывание).
2. Динамический. В этом случае используется наследование в сочетании с виртуальными функциями (позднее связывание).

Статический полиморфизм

«Перегрузка» функции – это объявление функции с тем же именем несколько раз. Чтобы компилятор мог отличать «перегруженные» функции между собой, эти функции **должны отличаться списком входных параметров**.

В общем случае, объявление перегруженной функции в некоторой области видимости выглядит следующим образом:

```
return_type1 FunName(parameters_list_1)
{
    // ...
}
return_type2 FunName(parameters_list_2)
{
    // ...
}
return_typeN FunName(parameters_list_N)
{
    // ...
}
```

Статический полиморфизм

```
1  #include <iostream>
2
3  bool _equal(char c1, char c2)
4  {
5      return static_cast<bool>(c1 == c2);
6  }
7
8  bool _equal(int i1, int i2)
9  {
10     return static_cast<bool>(i1 == i2);
11 }
12
13 bool _equal(double d1, double d2)
14 {
15     return static_cast<bool>(d1 == d2);
16 }
17
```

```
18  int main()
19  {
20      std::cout << _equal(4, 6) << " " <<
21          _equal('a', 'a') << " " <<
22          _equal(4.123, 4.2342) << std::endl;
23      return 0;
24  }
```

```
shulginaali@075-AShulgina:~/cpp$ ./a.out
0 1 0
shulginaali@075-AShulgina:~/cpp$
```

Пример перегрузки конструкторов в классе

```
1  #include <iostream>
2
3  class Contact
4  {
5  private:
6      int mobile_number;
7      int home_number;
8
9  public:
10     Contact()
11     {
12         std::cout << "default constructor" << std::endl;
13         mobile_number = 12345678;
14         home_number = 87654321;
15     }
16     Contact(const int mob, const int home)
17     {
18         std::cout << "constructor with parameters" << std::endl;
19         mobile_number = mob;
20         home_number = home;
21     }
22     Contact(const Contact& contact)
23     {
24         std::cout << "copy constructor" << std::endl;
25         mobile_number = contact.mobile_number;
26         home_number = contact.home_number;
27     }
```

```
28     void print_numbers()
29     {
30         std::cout << "Mobile number: " << mobile_number;
31         std::cout << ", home number: " << home_number << std::endl;
32     }
33 };
```

```
34
35 int main()
36 {
37     Contact Tony;
38     std::cout << "Tony" << std::endl;
39     Tony.print_numbers();
40
41     Contact Bob = Tony;
42     std::cout << "Bob" << std::endl;
43     Bob.print_numbers();
44
45     Contact Alice(2346677, 8876512);
46     std::cout << "Alice" << std::endl;
47     Alice.print_numbers();
48     return 0;
49 }
```

Списки инициализации членов класса

```
1  #include <iostream>
2
3  class Point2d
4  {
5  private:
6      float x;
7      float y;
8
9  public:
10     Point2d()
11     {
12         std::cout << "default constructor" << std::endl;
13         x = 0.f;
14         y = 0.f;
15     }
16 };
```

В данном примере в конструкторе используется присваивание, а не инициализация.

Списки инициализации членов класса

Если в классе Point2d появится константное поле, то его необходимо будет проинициализировать либо при создании объекта, либо через список инициализации.

Поскольку разрешено инициализировать константные переменные (но не присваивать им значения!).

```
3  class Point2d
4  {
5  private:
6      float x;
7      float y;
8      const int h;
9
10 public:
11     Point2d() : x(0.f), y(0.f), h(0)
12     {
13         std::cout << "default constructor" << std::endl;
14     }
15     Point2d(const double xx, const double yy, const int hh) :
16         x(xx), y(yy), h(hh)
17     {
18         std::cout << "constructor with parameters" << std::endl;
19     }
20     void print()
21     {
22         std::cout << x << " " << y << " " << h << std::endl;
23     }
24 };
25
```

Списки инициализации членов класса

Списки инициализации не только позволяют инициализировать константы, но и работают эффективнее (не выполняется копирующая инициализация).

Порядок в списке инициализации определяется списком деклараций.

```
3  class Point2d
4  {
5  private:
6      float x;
7      float y;
8      const int h;
9
10 public:
11     Point2d() : x(0.f), y(0.f), h(0)
12     {
13         std::cout << "default constructor" << std::endl;
14     }
15     Point2d(const double xx, const double yy, const int hh) :
16         x(xx), y(yy), h(hh)
17     {
18         std::cout << "constructor with parameters" << std::endl;
19     }
20     void print()
21     {
22         std::cout << x << " " << y << " " << h << std::endl;
23     }
24 };
25
```

Списки инициализации членов класса

```
3  class Point2d
4  {
5  private:
6      float xy[2];
7      const int h;
8
9  public:
10     Point2d() : xy{0.f, 0.f}, h(0)
11     {
12         std::cout << "default constructor" << std::endl;
13     }
14     Point2d(const double xx, const double yy, const int hh) :
15         xy{static_cast<float>(xx), static_cast<float>(yy)}, h(hh)
16     {
17         std::cout << "constructor with params" << std::endl;
18     }
19     void print()
20     {
21         std::cout << xy[0] << " " << xy[1] << " " << h << std::endl;
22     }
23 };
```

Если член класса — массив, то список инициализации приобретает следующий вид.

Задачи

1. Реализовать класс RGBA (конструкторы по умолчанию (черный цвет), с параметрами и копирования), хранящий в себе настройки цвета. Реализовать функции:
 - setColor(r, g, b, a = 255) – принимает все значения цвета и устанавливает их (alpha по умолчанию 255)
 - setColor() – не принимает параметров, устанавливает всем цветам значение 0
 - red(), green(), blue(), alpha() – возвращают значения конкретных цветов (не забыть про константность)
 - print(enum PrintType = E_RGBA) – печать на экран значений в соответствии с переданным типом (по умолчанию E_RGB)
 - E_RGBA: в строку (Пример: 255 255 255 255)
 - E_HEX: в hex-формате (Пример: `#ffffff`)
 - * E_INT: в числовом формате (int, побитовый сдвиг)
 - sum(const RGBA) – сложение цвета с переданным цветом, возвращает новый объект RGBA (компоненты в сумме не могут превысить 255).

Декомпонизировать реализацию на файлы, в функции main создать объект класса, протестировать все реализованные функции.

ООП: НАСЛЕДОВАНИЕ И ДИНАМИЧЕСКИЙ ПОЛИМОРФИЗМ

Семинар 3

Наследование

Наследование – это процесс, посредством которого один объект может приобретать свойства другого.

Наследование позволяет:

1. Создавать иерархии классов. Каждый класс может как использоваться сам по себе, так и служить основой для создания новой иерархии.
2. Описывать логически связанные типы без дублирования кода: все их определяющие общие черты ложатся в основу базового класса.

Наследование

В C++ есть несколько типов наследования:

- **public:** `public` члены базового класса становятся `public` членами в наследуемом классе, `protected` члены - аналогично;
- **protected:** `public` и `protected` члены становятся `protected` в наследуемом классе;
- **private:** `public` и `protected` члены становятся `private` в наследуемом классе.

Заметка: `private` члены базового класса недоступны в наследуемом

Наследование

```

1  #include <iostream>
2
3  using namespace std;
4
5  class Device {
6  public:
7      // constructor
8      Device() {
9          cout << "Device constructor called" << endl;
10     }
11     // destructor
12     ~Device() {
13         cout << "Device destructor called" << endl;
14     }
15 };
16
17 class Computer: public Device {
18 public:
19     Computer() {
20         cout << "Computer constructor called" << endl;
21     }
22     ~Computer() {
23         cout << "Computer destructor called" << endl;
24     }
25 };
26

```

```

27 class Laptop: public Computer {
28     public:
29         Laptop() {
30             cout << "Laptop constructor called" << endl;
31         }
32         ~Laptop() {
33             cout << "Laptop destructor called" << endl;
34         }
35 };
36
37 int main() {
38     cout << "\tConstructors" << endl;
39     Laptop Laptop_instance;
40     cout << "\tDestructors" << endl;
41     return 0;
42 }

```

```

shulginaali@075-AShulgina:~/cpp$ ./a.out
Constructors
Device constructor called
Computer constructor called
Laptop constructor called
Destructors
Laptop destructor called
Computer destructor called
Device destructor called
shulginaali@075-AShulgina:~/cpp$

```

Наследование

Конструкторы

вызываются в порядке иерархии наследования классов, деструкторы — в обратном порядке.

```
shulginaali@075-AShulgina:~/cpp$ ./a.out
    Constructors
Device constructor called
Computer constructor called
Laptop constructor called
    Destructors
Laptop destructor called
Computer destructor called
Device destructor called
shulginaali@075-AShulgina:~/cpp$
```

Наследование

```
1 #include <iostream>
2 using namespace std;
3
4 class Device {
5     public:
6         int serial_number = 12345678;
7
8         void turn_on() {
9             cout << "Device is on" << endl;
10        }
11 };
12
13 class Computer: private Device {
14     public:
15         void say_hello() {
16             turn_on();
17             cout << "Welcome to Windows!" << endl;
18        }
19 };
20
```

Раннее и позднее связывание

- раннее связывание – использование перегруженных функций, или статический полиморфизм;
- позднее связывание – использование наследование в сочетании с виртуальными функциями.

Виртуальная функция – это функция, объявляемая в базовом классе и переопределяемая в производном классе. Производный класс по своему усмотрению реализует виртуальную функцию. Чтобы объявить виртуальную функцию, используется ключевое слово `virtual`.

Виртуальные функции

В языке C++ в унаследованном классе, виртуальная функция, которая переопределяет одноименную функцию базового класса, может быть обозначена с помощью спецификатора `override`.

```
1  class BaseClass
2  {
3      virtual void funcA();
4      virtual void funcB() const;
5      virtual void funcC(int = 0);
6      void funcD();
7  };
8
9  class DerivedClass: public BaseClass
10 {
11     virtual void funcA(); // ok, works as intended
12
13     virtual void funcB(); // DerivedClass::funcB() is non-const, so it does not
14                          // override BaseClass::funcB() const and it is a new member function
15
16     virtual void funcC(double = 0.0); // DerivedClass::funcC(double) has a different
17                                      // parameter type than BaseClass::funcC(int), so
18                                      // DerivedClass::funcC(double) is a new member function
19 };
```

Виртуальные функции

```
1  class BaseClass
2  {
3      virtual void funcA();
4      virtual void funcB() const;
5      virtual void funcC(int = 0);
6      void funcD();
7  };
8
9  class DerivedClass: public BaseClass
10 {
11     virtual void funcA() override; // ok
12
13     virtual void funcB() override; // compiler error: DerivedClass::funcB() does not
14                                     // override BaseClass::funcB() const
15
16     virtual void funcC( double = 0.0 ) override; // compiler error:
17                                                     // DerivedClass::funcC(double) does not
18                                                     // override BaseClass::funcC(int)
19
20     void funcD() override; // compiler error: DerivedClass::funcD() does not
21                             // override the non-virtual BaseClass::funcD()
22 };
23
```

Правило: использовать спецификатор `override` при переопределении виртуальных функций

Виртуальные функции

Для обозначения функций, которые нельзя переопределить, а также классов от которых нельзя наследоваться, может быть использован спецификатор `final`.

```
1  class BaseClass
2  {
3      virtual void func() final;
4  };
5
6  class DerivedClass: public BaseClass
7  {
8      virtual void func(); // compiler error: attempting to
9                          // override a final function
10 };
11
```

```
1  class BaseClass final
2  {
3  };
4
5  class DerivedClass: public BaseClass // compiler error: BaseClass is
6                                     // marked as non-inheritable
7  {
8  };
9
```

Пример

```

1  #include <iostream>
2
3  // Базовый класс
4  class Base
5  {
6  public:
7      // virtual - признак виртуальной функции
8      virtual void PrintInfo()
9      {
10         std::cout << "Base" << std::endl;
11     }
12 };
13
14 // Класс, унаследованный от класса Base,
15 // важно: здесь должен быть модификатор public
16 class Derived : public Base
17 {
18 public:
19     virtual void PrintInfo() override
20     {
21         std::cout << "Derived" << std::endl;
22     }
23 };

```

```

24
25 int main(int argc, char** argv)
26 {
27     // 1. Создать экземпляры базового и производного класса
28     Base obj1;
29     Derived obj2;
30
31     // 2. Объявить указатель на базовый класс
32     Base* p;
33
34     // 3. Использовать правило: указатель на базовый класс может указывать
35     //     на любой экземпляр базового и производного от него класса.
36     //     Ниже демонстрируется полиморфизм.
37     // 3.1. Установить указатель p на экземпляр базового класса obj1
38     //     и вызвать PrintInfo()
39     p = &obj1;
40     p->PrintInfo(); // Base
41
42     // 3.2. Установить указатель p на экземпляр производного класса
43     //     и вызвать PrintInfo()
44     p = &obj2;
45     p->PrintInfo(); // Derived - это есть полиморфизм (слово virtual)
46     return 0;
47 }

```

```

shulginaali@075-AShulgina:~/cpp$ ./a.out
Base
Derived

```

Особенности

Относительно парадигмы классов и объектов для полиморфизма можно выделить следующие характерные особенности:

- полиморфизм не характеризует объект;
- реализацию полиморфизма определяет архитектура класса;
- полиморфизм является характеристикой функций-членов класса;
- весь класс не может быть полиморфным, полиморфными могут быть только функции члены класса.

Случаи реализации полиморфизма

Если в иерархии классов реализованы виртуальные функции, то полиморфизм реализуется в следующих случаях:

- при объявлении указателя (*) на базовый класс и вызове виртуальной функции соответствующего экземпляра класса, являющегося частью иерархии;
- при передаче указателя (*) на базовый класс в некоторую функцию, вызывающую виртуальную функцию базового класса;
- при передаче ссылки (&) на базовый класс в некую функцию, вызывающую виртуальную функцию базового класса.

Вызов виртуальной функции по указателю (->)

```

3 // Базовый класс
4 class Base
5 {
6 public:
7     // virtual - признак виртуальной функции
8     virtual void PrintInfo()
9     {
10         std::cout << "Base" << std::endl;
11     }
12 };
13
14 // Класс, унаследованный от класса Base,
15 // важно: здесь должен быть модификатор public
16 class Derived : public Base
17 {
18 public:
19     virtual void PrintInfo() override
20     {
21         std::cout << "Derived" << std::endl;
22     }
23 };

```

```

25 int main(int argc, char** argv)
26 {
27     // 1. Создать экземпляры базового и производного класса
28     Base obj1;
29     Derived obj2;
30
31     // 2. Объявить указатель на базовый класс
32     Base* p;
33
34     // 3. Использовать правило: указатель на базовый класс может указывать
35     //     на любой экземпляр базового и производного от него класса.
36     //     Ниже демонстрируется полиморфизм.
37     // 3.1. Установить указатель p на экземпляр базового класса obj1
38     //     и вызвать PrintInfo()
39     p = &obj1;
40     p->PrintInfo(); // Base
41
42     // 3.2. Установить указатель p на экземпляр производного класса
43     //     и вызвать PrintInfo()
44     p = &obj2;
45     p->PrintInfo(); // Derived - это есть полиморфизм (слово virtual)
46     return 0;
47 }

```

chernousov@redalbook-pro:~/misp-2/examples/004\$./a.out

Base

Derived

Вызов виртуальной функции через ссылку (&) на базовый класс

```

3 // Класс A - базовый класс в иерархии A <= B <= C
4 class A
5 {
6 public:
7     virtual void Show()
8     {
9         std::cout << "A::Show()" << std::endl;
10    }
11 };
12
13 // Класс B - унаследованный от класса A
14 class B : public A
15 {
16 public:
17     // виртуальный метод - переопределяет одноименный метод класса A
18     void Show() override
19     {
20         std::cout << "B::Show()" << std::endl;
21     }
22 };
23
24 // Класс C - унаследован от класса B
25 class C : public B
26 {
27 public:
28     // виртуальный метод - переопределяет одноименный метод класса B
29     void Show() override
30     {
31         std::cout << "C::Show()" << std::endl;
32     }
33 };

```

```

35 /* Функция, получающая ссылку на базовый класс A,
36    * в функции демонстрируется полиморфизм.
37    */
38 void DemoPolymorphism(A& ref)
39 {
40     /* Вызов виртуального метода, метод определяется при выполнении.
41        * Здесь неизвестно, метод какого класса A, B или C нужно вызвать.
42        */
43     ref.Show();
44 }
45
46 int main(int argc, char** argv)
47 {
48     // 1. объявить экземпляры классов A, B, C
49     A objA;
50     B objB;
51     C objC;
52
53     /* 2. Вызвать функцию DemoPolymorphism(),
54        * в зависимости от того, какой экземпляр передается,
55        * вызывается соответствующий метод класса.
56        */
57     DemoPolymorphism(objA); // A::Show()
58     DemoPolymorphism(objB); // B::Show()
59     DemoPolymorphism(objC); // C::Show()
60
61     return 0;
62 }

```

chernousov@redmibook-pro:~/nisp-2/examples/004\$./a.out

```

A::Show()
B::Show()
C::Show()

```

std::vector

- заголовочный файл `<vector>` – <https://en.cppreference.com/w/cpp/container/vector>
- вектор – контейнер, инкапсулирующий массивы динамического размера
- `std::vector<T>`, `T` некий тип данных $\Rightarrow$ вектор – шаблон класса для контейнеров последовательностей,
- хранит данные линейно
- обеспечивает быстрый случайный доступ к любому элементу

std::vector

Основные методы:

- `push_back(T)` – добавить элемент в конец вектора,
- `pop_back()` – снять последний элемент,
- `size()` – получить размер вектора,
- `at()/[]` – доступ к элементу.

std::vector

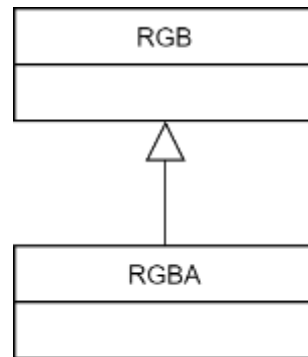
```
1  #include <iostream>
2  #include <vector>
3
4  int main(int argc, char** argv)
5  {
6      std::vector<int> vec{1, 2, 3, 4, 5};
7      vec.push_back(6);
8      vec.push_back(7);
9      std::cout << "Size: " << vec.size() << " " <<
10         "at: " << vec.at(vec.size() - 1) << " " <<
11         "[]: " << vec[vec.size() - 1] << std::endl;
12      vec.pop_back();
13      for (size_t i = 0; i < vec.size(); ++i)
14      {
15          vec[i] *= 3;
16          std::cout << vec[i] << " ";
17      }
18      std::cout << std::endl;
19      return 0;
20 }
```

```
chernousov@redmibook-pro:~/misp-2/examples/006$ ./a.out
Size: 7 at: 7 []: 7
3 6 9 12 15 18
```

Задача

Переписать задачу прошлого семинара с использованием следующей структуры.

Класс RGBA должен переопределять виртуальные методы класса RGB, а также перегружать некоторые функции.



Создать в функции `main()` `vector` указателей на базовый класс, записать в него несколько экземпляров классов RGB/RGBA. Вызвать функцию `print()` для каждого элемента контейнера.